

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Ingeniería en Sistemas Computacionales

Programa Máquina de Turing

Alumno: Linares Ojeda Carlos Elias

No. de Boleta: 2024630337

Grupo: 4CM2

Unidad de Aprendizaje: Teoría de la Computación

Profesor: Genaro Juárez Martínez



Ciudad de México, June 22, 2025

Índice

1. Introducción	2
2. Marco teórico	2
2.1. Lenguaje objetivo	2
2.2. Definición formal	2
2.3. Tabla de transiciones	3
2.4. Descripciones instantáneas	4
3. Desarrollo	4
3.1. Programa Máquina de Turing	4
3.2. Planteamiento	4
3.3. Funcionamiento del algoritmo	5
4. Análisis de complejidad	6
4.1. Complejidad temporal	6
4.2. Complejidad espacial	7
5. Implementación en Java	7
5.1. Arquitectura del programa	7
5.2. La cinta	9
5.3. La función de transición	9
5.4. El motor de simulación	10
5.5. La interfaz gráfica	10
5.6. Modos de ejecución	11
5.7. Registro de la computación	12
5.8. Alcance de la versión actual	13
6. Resultados	13
6.1. Cadena aceptada en modo automático	13
6.2. Cadena rechazada	15
6.3. Cadena de 10 caracteres	16
6.4. Ejecución en modo no gráfico	16
6.5. Archivo de registro	17
7. Conclusión	18
8. Referencias bibliográficas	19

1. Introducción

Este reporte presenta la implementación de una Máquina de Turing en Java que reconoce el lenguaje formal $L = \{0^n 1^n \mid n \geq 1\}$. La implementación está basada en el ejercicio 8.2 de la segunda edición del libro de John Hopcroft, Rajeev Motwani y Jeffrey Ullman, *Introduction to Automata Theory, Languages, and Computation* [1].

En esta versión, el programa está organizado en componentes independientes: un motor de simulación que ejecuta la máquina a partir de su definición formal y una interfaz gráfica que se encarga de mostrar lo que ocurre en cada paso. La Máquina de Turing implementada incluye:

- Interfaz gráfica desarrollada en Java Swing que muestra la cinta, la posición del cabezal, el estado actual y la descripción instantánea de cada configuración.
- Ejecución manual, paso a paso, y ejecución automática animada que se puede pausar.
- Animación de la máquina para cadenas de hasta 10 caracteres.
- Modo no gráfico para cadenas de más de 10 caracteres, que se ejecuta en segundo plano sin congelar la ventana.
- Registro de la computación en un archivo de texto.
- Detección de la aceptación o el rechazo de la cadena, con un límite de pasos como protección contra ciclos infinitos.

2. Marco teórico

Una Máquina de Turing está formada por un control finito con un conjunto de estados, una cinta infinita dividida en celdas y un cabezal que, en cada movimiento, lee el símbolo de una celda, escribe otro símbolo en su lugar, cambia de estado y se desplaza una celda a la izquierda o a la derecha [1, 2, 4].

2.1. Lenguaje objetivo

El lenguaje $L = \{0^n 1^n \mid n \geq 1\}$ consiste en todas las cadenas que tienen:

- Un número igual de ceros y unos.
- Todos los ceros antes que todos los unos.
- Al menos un cero y un uno ($n \geq 1$).

Ejemplos de cadenas válidas: 01, 0011, 000111, 00001111. Ejemplos de cadenas que no pertenecen a L : 0, 10, 001, 011, 0101, 1100 y la cadena vacía ε .

2.2. Definición formal

La Máquina de Turing se define como $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ donde:

- $Q = \{q_0, q_1, q_2, q_3, q_4\}$ (conjunto de estados)
- $\Sigma = \{0, 1\}$ (alfabeto de entrada)

- $\Gamma = \{0, 1, X, Y, B\}$ (alfabeto de la cinta)
- q_0 es el estado inicial
- B es el símbolo en blanco
- $F = \{q_4\}$ (conjunto de estados de aceptación)
- δ es la función de transición definida en la Tabla 1

Los símbolos X y Y son marcas: X indica un 0 que ya fue emparejado y Y un 1 que ya fue emparejado.

2.3. Tabla de transiciones

Cada entrada $\delta(q, a) = (p, b, D)$ indica que, si la máquina está en el estado q y el cabezal lee el símbolo a , entonces escribe b en esa celda, pasa al estado p y mueve el cabezal una celda en la dirección D (L a la izquierda, R a la derecha). Un guion indica que la transición no está definida: si la máquina llega a esa combinación, se detiene. La Figura 1 muestra la misma función como diagrama de transiciones.

Tabla 1: Función de transición δ de la Máquina de Turing.

Estado	0	1	X	Y	B
q_0	(q_1, X, R)	—	—	(q_3, Y, R)	—
q_1	$(q_1, 0, R)$	(q_2, Y, L)	—	(q_1, Y, R)	—
q_2	$(q_2, 0, L)$	—	(q_0, X, R)	(q_2, Y, L)	—
q_3	—	—	—	(q_3, Y, R)	(q_4, B, R)
q_4	—	—	—	—	—

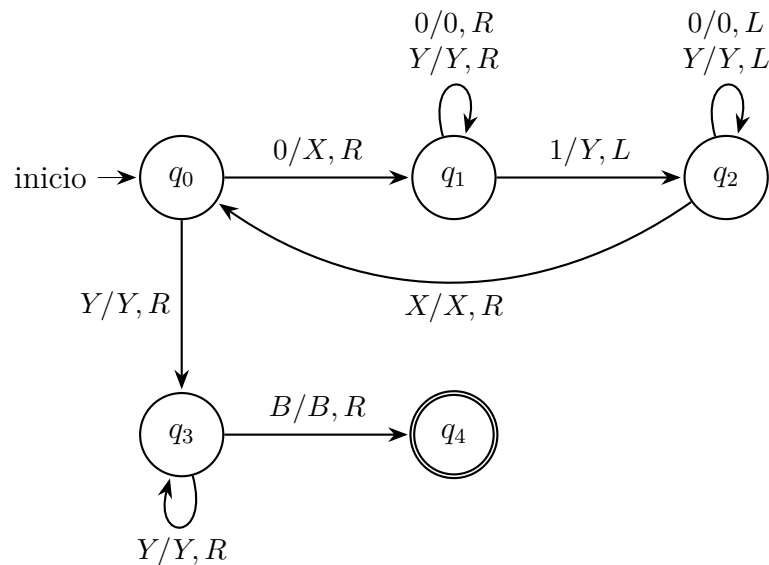


Figura 1: Diagrama de transiciones de la máquina. Una arista con la etiqueta $a/b, D$ indica que, al leer a , la máquina escribe b y mueve el cabezal en la dirección D . El estado q_4 (doble círculo) es el de aceptación.

2.4. Descripciones instantáneas

Una descripción instantánea (DI) resume la configuración completa de la máquina en un momento dado. Se escribe como $X_1 X_2 \cdots X_{i-1} q X_i X_{i+1} \cdots X_n$, donde q es el estado actual, $X_1 X_2 \cdots X_n$ es la porción de la cinta que contiene símbolos distintos del blanco y el cabezal está sobre X_i , el símbolo que aparece inmediatamente a la derecha del estado [1, 3].

Un movimiento de la máquina se denota con $\vdash$. Por ejemplo, como $\delta(q_0, 0) = (q_1, X, R)$, con la entrada 0011 se tiene

$$q_0 0011 \vdash X q_1 011.$$

La máquina acepta una cadena w si, partiendo de la DI inicial $q_0 w$, llega después de cero o más movimientos a una DI cuyo estado pertenece a F ; es decir, si $q_0 w \vdash^* \alpha q_4 \beta$ para algunas cadenas $\alpha, \beta \in \Gamma^*$. En la Sección 3.3 se muestra la secuencia completa de descripciones instantáneas para esta entrada.

3. Desarrollo

3.1. Programa Máquina de Turing

El programa de la máquina de Turing debe reconocer el lenguaje $L = \{0^n 1^n \mid n \geq 1\}$. La máquina de Turing se encuentra en el libro de John Hopcroft (ejercicio 8.2, segunda edición) y su tabla de transiciones es la Tabla 1. Los requisitos del programa son:

1. El programa debe recibir una cadena definida por el usuario o que sea determinada automáticamente por la máquina, una cadena de longitud como máximo de 1000 caracteres.
2. La salida del programa debe ser a un archivo de texto y utilizando descripciones instantáneas en cada paso de la computación.
3. Animar la máquina de Turing con cadenas menores o iguales a 10 caracteres.

3.2. Planteamiento

Existen diversas estrategias para simular una Máquina de Turing, pero una de las más comprensibles es construir el modelo directamente a partir de su definición formal: una cinta infinita dividida en celdas, un cabezal de lectura/escritura, un conjunto de estados y una función de transición que guía el comportamiento paso a paso.

En esta versión, cada uno de esos elementos se implementó como un componente independiente: la cinta, la función de transición, el motor que ejecuta la computación, el registro en archivo y la interfaz gráfica. El motor no depende de la ventana: recibe la cadena de entrada, ejecuta una transición cada vez que se le solicita e informa el resultado de ese paso (la máquina sigue en ejecución, aceptó o rechazó). Gracias a esta separación, el mismo motor se utiliza tanto en la animación paso a paso como en el modo no gráfico, y la interfaz se limita a mostrar la configuración actual.

El enfoque de la máquina se basa en recorrer la cadena marcando parejas: cada 0 se reemplaza por X y el primer 1 sin marcar que se encuentra a su derecha se reemplaza por Y ; después, el cabezal regresa para buscar el siguiente 0. Si al terminar solo quedan símbolos Y antes del primer blanco, la cadena tiene la forma $0^n 1^n$ y es aceptada.

La implementación se desarrolló en **Java** debido a su estructura orientada a objetos, que permite representar cada elemento de la definición formal con una clase, a su soporte

para interfaces gráficas (Swing) y a su facilidad para gestionar archivos y estructuras de datos dinámicas como listas y mapas.

3.3. Funcionamiento del algoritmo

La máquina funciona mediante el siguiente algoritmo:

1. **Estado q_0 :** busca el siguiente 0 sin marcar.
 - Si lee un 0, lo marca con X , se mueve a la derecha y pasa a q_1 .
 - Si lee una Y , ya no quedan ceros por marcar y pasa a q_3 (verificación final).
2. **Estado q_1 :** busca el primer 1 sin marcar.
 - Se mueve a la derecha saltando los 0 y las Y .
 - Cuando encuentra un 1, lo marca con Y , se mueve a la izquierda y pasa a q_2 .
3. **Estado q_2 :** regresa hacia la izquierda.
 - Se mueve a la izquierda saltando los 0 y las Y hasta encontrar la X más reciente.
 - Al leer la X se mueve a la derecha y regresa a q_0 para repetir el proceso con el siguiente 0.
4. **Estado q_3 :** verificación final.
 - Se mueve a la derecha mientras lea Y .
 - Si encuentra el símbolo en blanco, pasa al estado q_4 .
5. **Estado q_4 :** estado de aceptación; la máquina se detiene y acepta la cadena.

Si la máquina llega a una combinación de estado y símbolo sin transición definida, se detiene y rechaza la cadena. Esto ocurre en tres situaciones:

- q_0 lee un 1: no encuentra el 0 con el que debe comenzar el emparejamiento; por ejemplo, la cadena comienza con 1.
- q_1 lee un blanco: se marcó un 0 que no tiene un 1 correspondiente, es decir, hay más ceros que unos.
- q_3 lee un 0 o un 1: a la derecha de las Y quedan símbolos sin marcar, ya sea porque hay más unos que ceros o porque aparece un 0 después de un 1.

Como ejemplo, la Tabla 2 muestra la computación completa con la entrada 0011: cada fila indica la transición aplicada y la descripción instantánea resultante. La máquina realiza 13 movimientos y termina en el estado de aceptación q_4 .

Tabla 2: Computación de la máquina con la entrada 0011.

Paso	Transición aplicada	Descripción instantánea
0	—	q_0 0011
1	$\delta(q_0, 0) = (q_1, X, R)$	X q_1 011
2	$\delta(q_1, 0) = (q_1, 0, R)$	$X0$ q_1 11
3	$\delta(q_1, 1) = (q_2, Y, L)$	X q_2 0Y1
4	$\delta(q_2, 0) = (q_2, 0, L)$	q_2 X0Y1
5	$\delta(q_2, X) = (q_0, X, R)$	X q_0 0Y1
6	$\delta(q_0, 0) = (q_1, X, R)$	XX q_1 Y1
7	$\delta(q_1, Y) = (q_1, Y, R)$	XXY q_1 1
8	$\delta(q_1, 1) = (q_2, Y, L)$	XX q_2 YY
9	$\delta(q_2, Y) = (q_2, Y, L)$	X q_2 XYY
10	$\delta(q_2, X) = (q_0, X, R)$	XX q_0 YY
11	$\delta(q_0, Y) = (q_3, Y, R)$	XXY q_3 Y
12	$\delta(q_3, Y) = (q_3, Y, R)$	$XXYY$ q_3 B
13	$\delta(q_3, B) = (q_4, B, R)$	$XXYYB$ q_4 B

4. Análisis de complejidad

4.1. Complejidad temporal

Para una cadena aceptada $w = 0^n 1^n$ es posible contar exactamente los movimientos de la máquina. Cada iteración i , con $1 \leq i \leq n$, consta de:

- 1 movimiento en q_0 para marcar el i -ésimo 0 con X ;
- n movimientos en q_1 : atraviesa los $n - i$ ceros restantes y las $i - 1$ marcas Y ya escritas, y marca el i -ésimo 1;
- n movimientos en q_2 : regresa sobre esas mismas $i - 1$ marcas Y y $n - i$ ceros, y lee la X .

Es decir, cada iteración cuesta $2n + 1$ movimientos. Al terminar las n iteraciones, la verificación final cuesta $n + 1$ movimientos: uno en q_0 al leer la primera Y , $n - 1$ en q_3 para recorrer las Y restantes y uno más al leer el blanco. Por lo tanto, el número total de movimientos es

$$T(n) = n(2n + 1) + (n + 1) = 2n^2 + 2n + 1.$$

Como la longitud de la entrada es $m = 2n$, se tiene $T = \frac{m^2}{2} + m + 1$; es decir, la máquina realiza $O(m^2)$ movimientos. La Tabla 3 compara la fórmula con el número de pasos que reporta el programa al aceptar cada cadena: ambos valores coinciden.

Tabla 3: Número de pasos para cadenas aceptadas.

n	Cadena	Longitud m	$T(n) = 2n^2 + 2n + 1$	Pasos del programa
1	01	2	5	5
2	0011	4	13	13
3	000111	6	25	25
4	00001111	8	41	41
5	0000011111	10	61	61

Para las cadenas rechazadas la máquina se detiene antes, en la primera combinación sin transición. Como realiza a lo más una iteración por cada par marcado y cada iteración recorre $O(m)$ celdas, su tiempo también es $O(m^2)$.

4.2. Complejidad espacial

La máquina solo escribe sobre las celdas que ocupa la entrada (reemplaza los 0 por X y los 1 por Y) y, al final, avanza una celda más allá del primer blanco, por lo que utiliza a lo más $m + 2$ celdas de la cinta: su complejidad espacial es $O(m)$. En el programa, la cinta inicia con la cadena de entrada más algunos blancos adicionales y solo crece cuando el cabezal sale de ella, por lo que la memoria utilizada también es $O(m)$.

5. Implementación en Java

5.1. Arquitectura del programa

El programa está formado por seis clases y una enumeración. La Tabla 4 describe la responsabilidad de cada una y la Figura 2 muestra cómo se relacionan. La separación principal es entre la interfaz gráfica, que muestra la información y recibe las acciones del usuario, y el motor de simulación, que contiene toda la lógica de la máquina.

Tabla 4: Componentes del programa y sus responsabilidades.

Componente	Responsabilidad
TuringMachine	Ventana principal (Swing). Recibe y valida la cadena, controla los botones, dibuja la cinta y muestra el estado, la descripción instantánea y el resultado.
TuringMachineEngine	Motor de simulación. Mantiene la configuración actual (cinta, estado y número de paso) y ejecuta una transición a la vez.
Tape	Cinta: lista de símbolos y posición del cabezal. Lee, escribe y mueve el cabezal, agregando blancos cuando hace falta.
TransitionTable	Función de transición δ : guarda, para cada estado, la transición que corresponde a cada símbolo leído.
Transition	Una transición $\delta(q, a) = (p, b, D)$: estado siguiente, símbolo a escribir y dirección del movimiento.
StepStatus	Resultado de un paso: en ejecución, aceptada, rechazada o límite de pasos alcanzado.
SimulationLogger	Escribe la computación en el archivo <code>computing/simulacion_turing.txt</code> .

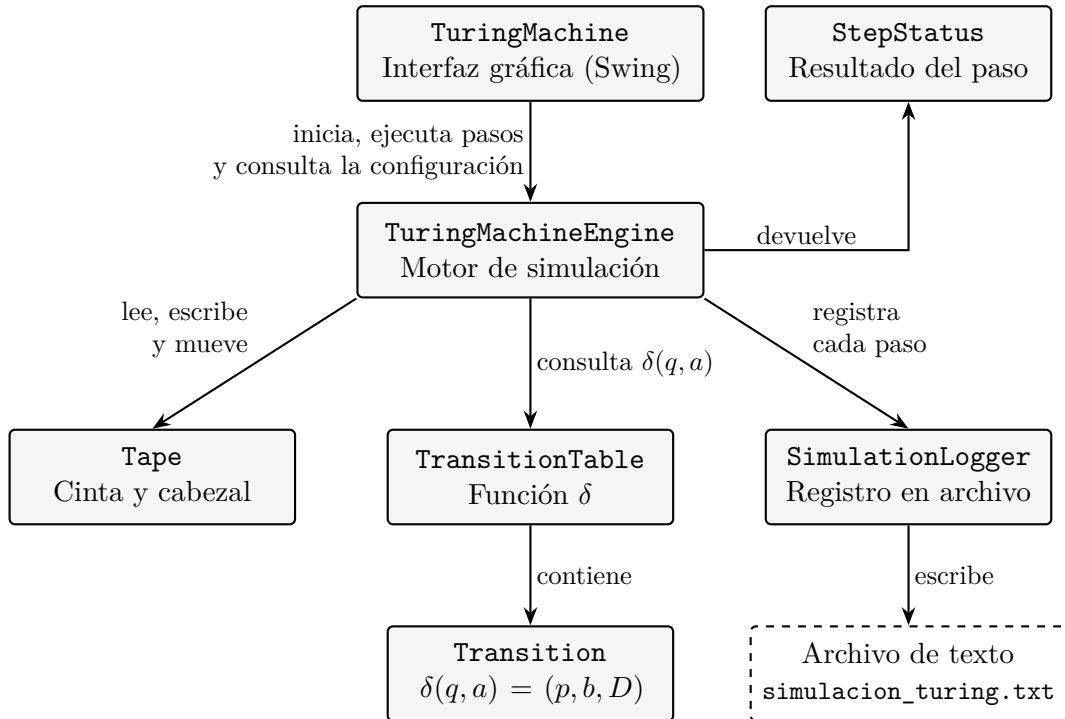


Figura 2: Relación entre las clases del programa. La interfaz solo se comunica con el motor; el motor es el único que modifica la cinta, consulta la función de transición y escribe el registro.

La Tabla 5 relaciona cada elemento de la definición formal $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ con su representación en el programa.

Tabla 5: Correspondencia entre la definición formal y el programa.

Elemento	Representación en el programa
Q	Cadenas de texto de q_0 a q_4 , que sirven como llaves de la tabla de transiciones.
q_0, F	Constantes del motor: el estado inicial es q_0 y el único estado de aceptación es q_4 .
Σ	La interfaz solo inicia la simulación si la cadena contiene únicamente ceros y unos.
Γ, B	Los símbolos de la cinta son caracteres; el blanco es el carácter B , que rellena la cinta alrededor de la entrada.
δ	La tabla de transiciones, con un objeto Transition por cada entrada definida en la Tabla 1.
Cabezal	Un índice entero que indica una posición dentro de la cinta.
DI	La configuración del motor (cinta, posición del cabezal y estado), que la interfaz muestra después de cada paso.

El programa no requiere bibliotecas externas, solo un JDK: se compila con `javac *.java` y se inicia con `java TuringMachine`.

5.2. La cinta

La clase **Tape** representa la cinta con una lista dinámica de caracteres (**ArrayList**) y un índice entero que indica la posición del cabezal. Al iniciar una simulación, la lista contiene los símbolos de la cadena de entrada seguidos de varios blancos: 10 en modo gráfico y, en modo no gráfico, el mayor entre 20 y la longitud de la cadena. El cabezal comienza en la primera celda, sobre el primer símbolo de la entrada.

La cinta ofrece cuatro operaciones: leer el símbolo que está bajo el cabezal, escribir un símbolo en esa celda y mover el cabezal una celda a la izquierda o a la derecha. Para simular una cinta infinita en ambas direcciones, si el cabezal avanza más allá del último símbolo se agrega un blanco al final de la lista, y si retrocede desde la primera celda se inserta un blanco al inicio y el cabezal queda en la posición 0. Así, la cinta solo crece cuando la computación lo necesita. Para dibujarla, la interfaz recibe una vista de solo lectura de la lista, de modo que únicamente el motor puede modificar la cinta.

5.3. La función de transición

Cada transición $\delta(q, a) = (p, b, D)$ se representa con un objeto inmutable de la clase **Transition**, que guarda el estado siguiente p , el símbolo a escribir b y la dirección D (L o R). La clase **TransitionTable** organiza estos objetos en un mapa de dos niveles: el primero se indexa por estado y el segundo por el símbolo leído, de modo que consultar $\delta(q, a)$ toma tiempo constante. La tabla se construye al crear el motor con las diez transiciones de la Tabla 1; el estado q_4 no tiene transiciones.

Cuando se consulta una combinación que no está en la tabla, la búsqueda devuelve un resultado vacío (**Optional**) en lugar de un valor nulo. El motor interpreta ese resultado como que no hay transición definida, lo que en una Máquina de Turing significa que la máquina se detiene sin aceptar.

5.4. El motor de simulación

La clase `TuringMachineEngine` guarda la configuración de la máquina —la cinta, el estado actual y el número de pasos— y su operación principal es ejecutar un paso. Cada paso sigue este ciclo:

1. Incrementa el contador de pasos y lee el símbolo a que está bajo el cabezal.
2. Consulta $\delta(q, a)$ en la tabla de transiciones. Si no está definida, la máquina se detiene, la cadena se rechaza y el paso devuelve `REJECTED`.
3. Registra en el archivo el paso y la transición que se va a aplicar (en modo no gráfico solo se registran algunos pasos; ver Sección 5.6).
4. Aplica la transición: escribe el nuevo símbolo, cambia de estado y mueve el cabezal.
5. Si el nuevo estado es q_4 , registra la aceptación, detiene la máquina y devuelve `ACCEPTED`.
6. Si el número de pasos supera el límite, lo registra, detiene la máquina y devuelve `LOOP_LIMIT`.
7. En cualquier otro caso devuelve `RUNNING` y la máquina queda lista para el siguiente paso.

El valor que devuelve cada paso pertenece a la enumeración `StepStatus`. La interfaz decide qué hacer únicamente con base en ese valor: mientras sea `RUNNING`, la simulación continúa; cualquier otro valor indica que la máquina se detuvo y que se debe mostrar el resultado.

El límite de pasos evita que una ejecución continúe indefinidamente. En modo gráfico es de 1000 pasos, muy por encima de los 61 movimientos que requiere la cadena aceptada más larga de este modo, 0000011111. En modo no gráfico, si m es la longitud de la entrada, la cinta comienza con $s = m + \text{máx}(20, m)$ celdas y el límite se calcula como

$$P_{\text{máx}} = \text{máx}(10000, 100s).$$

El motor comprueba este límite después de aplicar una transición y de comprobar si llegó a q_4 . Por eso una ejecución que se detiene por esta protección puede contabilizar $P_{\text{máx}} + 1$ pasos. En las cadenas rechazadas, el contador también incluye el intento final para el que no existe transición; ese intento no cambia la cinta ni mueve el cabezal.

5.5. La interfaz gráfica

La clase `TuringMachine` construye la ventana principal con Java Swing. La Figura 3 muestra sus partes.

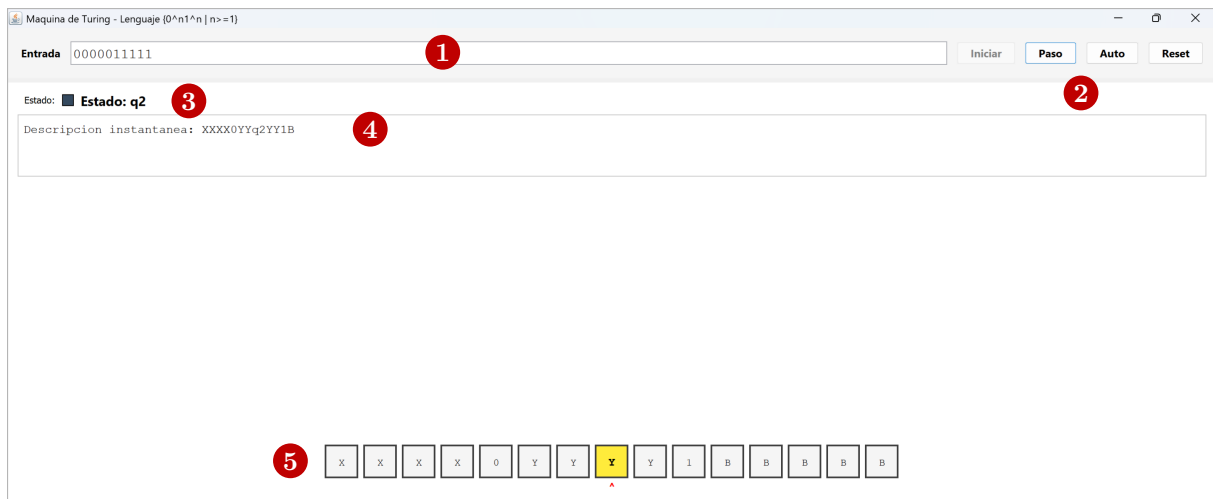


Figura 3: Ventana principal durante la ejecución manual de la cadena 0000011111, después del paso 39: (1) campo de entrada, (2) botones de control, (3) estado actual, (4) descripción instantánea y (5) cinta.

1. **Entrada:** campo donde el usuario escribe la cadena.
2. **Botones de control:** **Iniciar** comienza la simulación, **Paso** ejecuta una sola transición, **Auto** ejecuta la máquina de forma automática y **Reset** detiene la simulación y limpia la ventana.
3. **Estado actual:** un indicador de color y el nombre del estado. El indicador es gris antes de iniciar, gris azulado oscuro durante la simulación y verde cuando la máquina llega a q_4 .
4. **Descripción instantánea:** la configuración actual en la notación de la Sección 2.4. En la figura, XXXX0YYq2YY1B indica que la máquina está en q_2 y que el cabezal lee una Y. A la derecha del cabezal se muestran hasta siete símbolos.
5. **Cinta:** una fila de hasta 15 celdas alrededor del cabezal. La celda que lee el cabezal se resalta en amarillo y se señala con una marca $\wedge$ roja debajo de ella.

Al presionar **Iniciar**, el programa valida la cadena: si el campo está vacío o contiene símbolos distintos de 0 y 1, muestra un mensaje de error y no inicia la simulación. Si la cadena supera esta validación, elige el modo de ejecución según su longitud (Sección 5.6) y deshabilita **Iniciar** hasta que se presione **Reset**. Cuando la máquina se detiene, debajo de la descripción instantánea aparece el resultado: **Cadena aceptada (k pasos)** en verde, **Cadena rechazada** en rojo o, si se alcanzó el límite de pasos, **Cadena rechazada (Bucle infinito)**. En ese momento se deshabilitan **Paso** y **Auto**, y para probar otra cadena se presiona **Reset**.

5.6. Modos de ejecución

El modo de ejecución se elige automáticamente según la longitud de la cadena.

Modo gráfico (hasta 10 caracteres). La máquina se anima: después de cada paso se redibuja la cinta y se actualizan el estado, el indicador y la descripción instantánea. El usuario puede avanzar de dos formas:

- **Manual:** cada vez que se presiona **Paso** se ejecuta exactamente una transición.
- **Automático:** al presionar **Auto**, un temporizador de Swing ejecuta un paso cada 1.5 segundos y el botón cambia su texto a **Pausar**. Al presionarlo de nuevo la animación se detiene, y se puede continuar paso a paso o reanudarla.

En este modo todos los pasos se escriben en el archivo de registro.

Modo no gráfico (más de 10 caracteres).

- Al iniciar, un cuadro de diálogo avisa que la cadena se procesará en modo no gráfico y que los resultados se guardarán en el archivo de registro.
- La simulación se ejecuta en un hilo en segundo plano (**SwingWorker**), por lo que la ventana sigue respondiendo. Mientras tanto se muestra una barra de progreso, el indicador de estado se pone naranja y la cinta no se dibuja.
- Cada 100 pasos se informa el avance (número de paso y estado actual) en el recuadro de descripción.
- Los botones **Paso** y **Auto** se deshabilitan, ya que la ejecución es continua.
- Al terminar se muestran el número total de pasos y el resultado, y el indicador cambia a verde si la cadena se aceptó o a rojo si se rechazó.
- Para que el archivo no crezca demasiado con cadenas largas, se registran los primeros 50 pasos y, a partir de ahí, solo los pasos múltiplos de 10.

5.7. Registro de la computación

El registro se guarda en un archivo de texto. Su ubicación relativa al directorio desde donde se ejecuta el programa es:

`computing/simulacion_turing.txt`

La carpeta se crea automáticamente si no existe. El archivo se crea (o se sobrescribe) al iniciar cada simulación y contiene:

- Un encabezado con el lenguaje, la cadena de entrada, su longitud y el modo de ejecución.
- Por cada paso registrado: el número de paso, el estado actual, el símbolo leído y la transición aplicada, con el formato $\text{delta}(q, a) = (p, b, D)$.
- Si la máquina se detiene en un paso registrado porque no existe transición, el aviso correspondiente seguido de **Cadena rechazada**.
- Al aceptar: **Estado de aceptacion alcanzado**, **Cadena aceptada**, el total de pasos y la confirmación de que la cadena pertenece al lenguaje.
- Si se alcanza el límite de pasos, el mensaje **Límite de pasos alcanzado** con el valor del límite.

Cada paso se agrega al final del archivo en cuanto se ejecuta, por lo que lo registrado hasta ese momento se conserva aunque la ventana se cierre a mitad de la simulación. La Figura 10 muestra un ejemplo del archivo.

5.8. Alcance de la versión actual

El enunciado original pide entradas de hasta 1000 caracteres, la posibilidad de generar una cadena automáticamente y un archivo con las descripciones instantáneas de todos los pasos. La versión actual permite ingresar la cadena manualmente y valida que no esté vacía y que solo contenga ceros y unos; no incluye un generador automático ni una comprobación del máximo de 1000 caracteres. El modo de ejecución se selecciona según la longitud, sin que el usuario tenga que configurarlo.

Las descripciones instantáneas se muestran en la interfaz en modo gráfico; el archivo guarda estados, símbolos leídos y transiciones, sin incluir la cinta completa ni la posición del cabezal. En modo no gráfico, además, el registro es parcial después de los primeros 50 pasos. La aceptación y el aviso de límite siempre se escriben, pero el aviso de rechazo por falta de transición solo se escribe cuando el paso cumple la condición de registro.

El límite de pasos también puede detener una cadena que sí pertenece al lenguaje. Por ejemplo, $0^{200}1^{200}$ necesita 80401 movimientos, mientras que su límite es 80000 pasos. Aunque la interfaz muestra **Cadena rechazada (Bucle infinito)**, este resultado indica que se agotó el presupuesto de pasos; no demuestra un ciclo infinito ni equivale a un rechazo por la función de transición. Por ello, en el reporte se distinguen la aceptación, el rechazo por falta de transición y la detención por límite.

6. Resultados

Se realizaron pruebas con cadenas que pertenecen al lenguaje, como 01, 0011, 000111, 00001111 y 0000011111, y con cadenas que no pertenecen, como 0, 10, 001, 011, 0101, 1100 y 00011, para comprobar que la máquina acepta y rechaza correctamente. A continuación se muestran capturas del funcionamiento.

También se verificaron exhaustivamente las 2046 cadenas binarias no vacías de longitud entre 1 y 10. Para cada cadena se comparó el resultado del motor con el criterio de pertenencia: un bloque de ceros seguido de un bloque de unos con igual cantidad de ambos símbolos. Todos los resultados coincidieron y ninguna de estas ejecuciones alcanzó el límite de pasos.

6.1. Cadena aceptada en modo automático

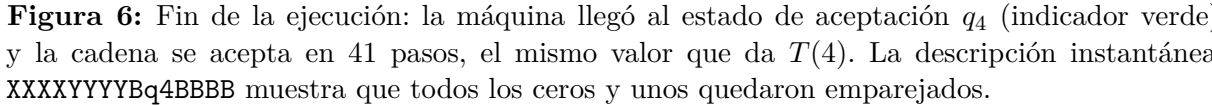
Las Figuras 4 a 6 muestran la ejecución automática de la cadena 00001111 ($n = 4$). Durante la animación, el botón **Auto** cambia a **Pausar**, y en cada paso se actualizan la cinta, el estado y la descripción instantánea.



Figura 4: Entrada 00001111 después del paso 2. La máquina ya marcó el primer 0 con X y en q_1 avanza hacia la derecha buscando un 1; el cabezal lee el tercer 0 de la entrada ($X0q1001111$).



Figura 5: La misma ejecución después del paso 4: el cabezal llegó al primer 1 ($X000q11111$), que en el siguiente paso se marcará con Y .



La Figura 7 muestra el rechazo de la cadena 00011, que tiene más ceros que unos.

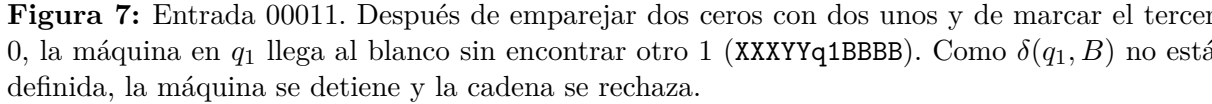


Tabla 6: Configuración de paro de algunas cadenas rechazadas.

Cadena	DI de paro	Transición faltante	Motivo
0	$X q_1 B$	$\delta(q_1, B)$	Hay más ceros que unos.
10	$q_0 10$	$\delta(q_0, 1)$	La cadena comienza con 1.
001	$XXY q_1 B$	$\delta(q_1, B)$	Hay más ceros que unos.
011	$XY q_3 1$	$\delta(q_3, 1)$	Hay más unos que ceros.
0101	$XY q_3 01$	$\delta(q_3, 0)$	Aparece un 0 después de un 1.
0110	$XY q_3 10$	$\delta(q_3, 1)$	Misma cantidad de ceros y unos, pero un 0 aparece después de los unos.
1100	$q_0 1100$	$\delta(q_0, 1)$	La cadena comienza con 1.
00011	$XXXXYY q_1 B$	$\delta(q_1, B)$	Hay más ceros que unos (Figura 7).

6.3. Cadena de 10 caracteres

La Figura 3 (Sección 5.5) muestra la ejecución manual de 0000011111, la cadena más larga que se anima, después del paso 39: la máquina acaba de marcar el cuarto 1 con Y y regresa en q_2 hacia la izquierda para buscar el siguiente 0. La Figura 8 muestra el final de esa ejecución.

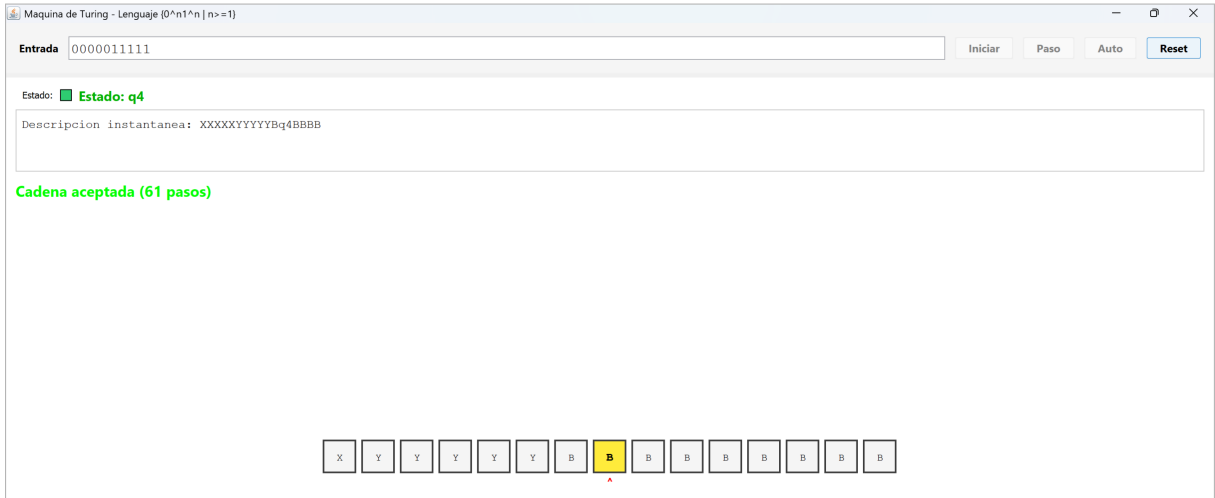


Figura 8: La cadena 0000011111 ($n = 5$) se acepta en 61 pasos, el mismo valor que da $T(5)$.

6.4. Ejecución en modo no gráfico

La entrada 000000111111 tiene 12 caracteres, por lo que se procesa en modo no gráfico. El mismo motor llega a q_4 en 85 pasos, coincidiendo con $T(6) = 2(6)^2 + 2(6) + 1$. La Figura 9 muestra la interfaz al finalizar: los controles **Paso** y **Auto** están deshabilitados y se muestra el resultado sin dibujar las celdas de la cinta.

Entrada

Estado: ■ Estado: q4

Simulación completada en 85 pasos.

Cadena aceptada (85 pasos)

Procesando en modo no grafico

Figura 9: Resultado de la entrada 000000111111 en modo no gráfico: estado q_4 y aceptación en 85 pasos.

Para comprobar la detención por límite también se ejecutó $0^{200}1^{200}$. El motor devolvió el resultado de límite alcanzado después de 80001 pasos, antes de los 80401 que necesita para aceptar. Esto confirma la diferencia entre detener una simulación por presupuesto de pasos y rechazar una cadena por falta de transición.

6.5. Archivo de registro

Además de la animación, la computación queda registrada en el archivo de texto. La Figura 10 muestra el archivo generado para la entrada 00000111111: el encabezado con los datos de la simulación, la transición aplicada en cada paso y, al final, el mensaje de aceptación con el total de 61 pasos, el mismo valor que muestra la interfaz en la Figura 8.

```

Calculos Maquina de Turing
Lenguaje: {0^n1^n | n>=1}
Cadena de entrada: 0000011111
Longitud: 10 caracteres
Modo: Grafico
Paso 1:
Estado: q0, Simbolo: '0'
delta(q0, 0) = (q1, X, R)

Paso 2:
Estado: q1, Simbolo: '0'
delta(q1, 0) = (q1, 0, R)

Paso 3:
Estado: q1, Simbolo: '0'
delta(q1, 0) = (q1, 0, R)

Paso 4:
Estado: q1, Simbolo: '0'
delta(q1, 0) = (q1, 0, R)

Paso 5:
Estado: q1, Simbolo: '0'
delta(q1, 0) = (q1, 0, R)

Paso 6:
Estado: q1, Simbolo: '1'
delta(q1, 1) = (q2, Y, L)

Paso 7:
Estado: q2, Simbolo: '0'
delta(q2, 0) = (q2, 0, L)

Paso 8:
Estado: q2, Simbolo: '0'
delta(q2, 0) = (q2, 0, L)

Paso 9:
Estado: q2, Simbolo: '0'
delta(q2, 0) = (q2, 0, L)

Paso 10:
Estado: q2, Simbolo: '0'

```

(a) Encabezado y primeros pasos.

```

delta(q2, Y) = (q2, Y, L)

Paso 53:
Estado: q2, Simbolo: 'Y'
delta(q2, Y) = (q2, Y, L)

Paso 54:
Estado: q2, Simbolo: 'Y'
delta(q2, Y) = (q2, Y, L)

Paso 55:
Estado: q2, Simbolo: 'X'
delta(q2, X) = (q0, X, R)

Paso 56:
Estado: q0, Simbolo: 'Y'
delta(q0, Y) = (q3, Y, R)

Paso 57:
Estado: q3, Simbolo: 'Y'
delta(q3, Y) = (q3, Y, R)

Paso 58:
Estado: q3, Simbolo: 'Y'
delta(q3, Y) = (q3, Y, R)

Paso 59:
Estado: q3, Simbolo: 'Y'
delta(q3, Y) = (q3, Y, R)

Paso 60:
Estado: q3, Simbolo: 'Y'
delta(q3, Y) = (q3, Y, R)

Paso 61:
Estado: q3, Simbolo: 'B'
delta(q3, B) = (q4, B, R)

Estado de aceptacion alcanzado
Cadena aceptada
Total de pasos: 61
La cadena pertenece al lenguaje {0^n1^n | n>=1}

```

(b) Últimos pasos y resultado.

Figura 10: Archivo `computing/simulacion_turing.txt` generado para la entrada 0000011111.

7. Conclusión

En esta versión del programa separé el motor de simulación de la interfaz gráfica, de modo que cada elemento de la definición formal (la cinta, la función de transición y los estados) quedó representado por su propio componente. Esto facilitó comprobar que la máquina se comporta exactamente como indica su tabla de transiciones; por ejemplo, el número de pasos que reporta el programa para las cadenas aceptadas coincide con el valor teórico $2n^2 + 2n + 1$.

Me gustó la práctica. Fue la que más esperé en todo el curso, ya que la máquina de Turing no solamente es un modelo teórico, considero que es un cambio de paradigma, una forma de ver a la computación. Me pareció increíble el trabajo de Turing. Sin duda dejó un legado muy fuerte a la computación, inteligencia artificial, matemáticas, etc. Siempre quise programar la máquina de Turing, pero consideraba que tenía que conocer su funcionamiento formal para poderlo hacer. Concluyo con que me gustó mucho la materia, y considero que me ayudó a ver la computación desde otro enfoque.

8. Referencias bibliográficas

- [1] J. E. Hopcroft, R. Motwani y J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*, segunda edición. Addison-Wesley, 2001.
- [2] Máquinas de Turing.
<https://www.matem.unam.mx/~omar/computabilidad/tema1.html>
- [3] J. D. Ullman.
<http://infolab.stanford.edu/~ullman/ialc/spr10/spr10.html#LECTURE>
- [4] Presentación. <https://ccc.inaoep.mx/~emorales/Cursos/Automatas/IntroMaquinasTuring.pdf>